

本节内容

算法

基本概念

知识总览

算法的基本概念

什么是算法

算法的五个特性

"好"算法的特质

什么是算法？

程序 = 数据结构 + 算法

如何用数据正确地描述现实世界的问题，并存入计算机

如何高效地处理这些数据，以解决实际问题

算法（Algorithm）是**对特定问题求解步骤的一种描述**，它是指令的有限序列，其中的每条指令表示一个或多个操作

要解决的问题：做番茄炒蛋

食材

- 鸡蛋 4个
- 西红柿 2个
- 料酒 少许
- 盐 1勺
- 糖 少许

步骤：

1. 西红柿切块
2. 鸡蛋加料酒打匀
3. 将锅烧热，倒入鸡蛋翻炒
4. 倒入西红柿翻炒
5. 加少许盐、糖
6. 装盘

什么是算法？

程序 = 数据结构 + 算法

求解问题的步骤

• 马云 • 4377亿元 • 56岁	• 马化腾 • 3683亿元 • 49岁	• 黄峥 • 2041亿元 • 40岁	• 丁磊 • 1781亿元 • 49岁	• 咸鱼 • 1039元 • 18岁
1	2	3	4	5

算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

什么是算法？

程序 = 数据结构 + 算法

求解问题的
步骤

- 马云
- 4377亿元
- 56岁

1

- 马化腾
- 3683亿元
- 49岁

2

- 黄峥
- 2041亿元
- 40岁

3

- 丁磊
- 1781亿元
- 49岁

4

- 咸鱼
- 1039元
- 18岁

5

算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

什么是算法？

程序 = 数据结构 + 算法

求解问题的
步骤

• 咸鱼 • 1039元 • 18岁	• 马云 • 4377亿元 • 56岁	• 马化腾 • 3683亿元 • 49岁	• 黄峥 • 2041亿元 • 40岁	• 丁磊 • 1781亿元 • 49岁
1	2	3	4	5

算法：将该线性表按照年龄递增排序

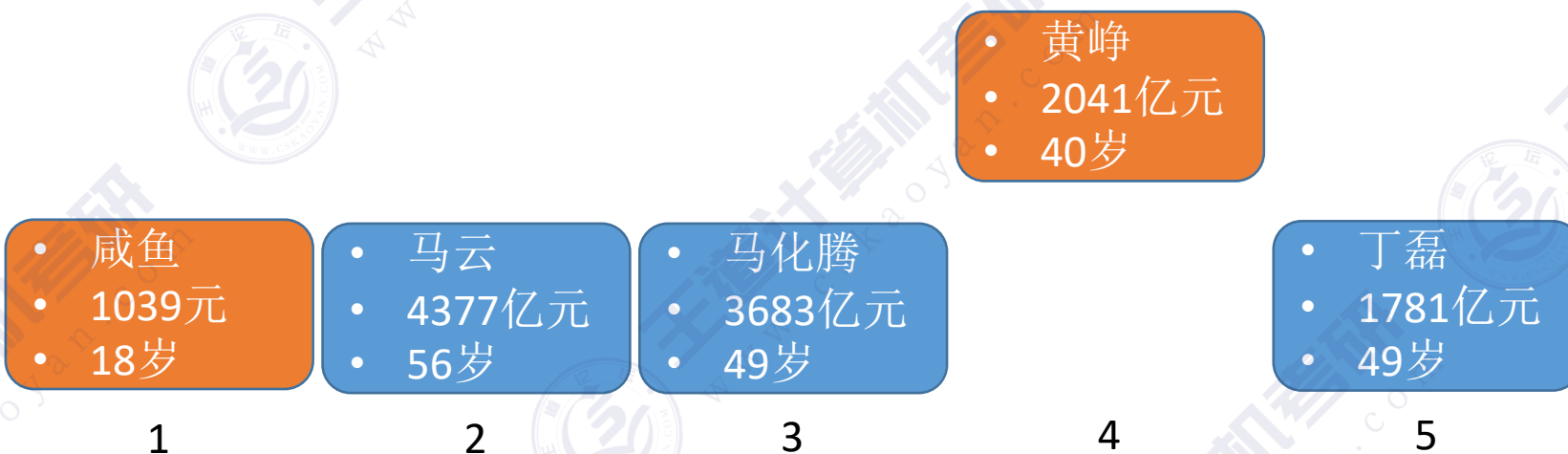
Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

Step 2: 扫描剩下的4个元素，找到年龄最小的一个元素，插入到第2个位置

什么是算法？

程序 = 数据结构 + 算法

求解问题的
步骤



算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

Step 2: 扫描剩下的4个元素，找到年龄最小的一个元素，插入到第2个位置

什么是算法？

程序 = 数据结构 + 算法

求解问题的
步骤

• 咸鱼 • 1039元 • 18岁	• 黄峥 • 2041亿元 • 40岁	• 马云 • 4377亿元 • 56岁	• 马化腾 • 3683亿元 • 49岁	• 丁磊 • 1781亿元 • 49岁
1	2	3	4	5

算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

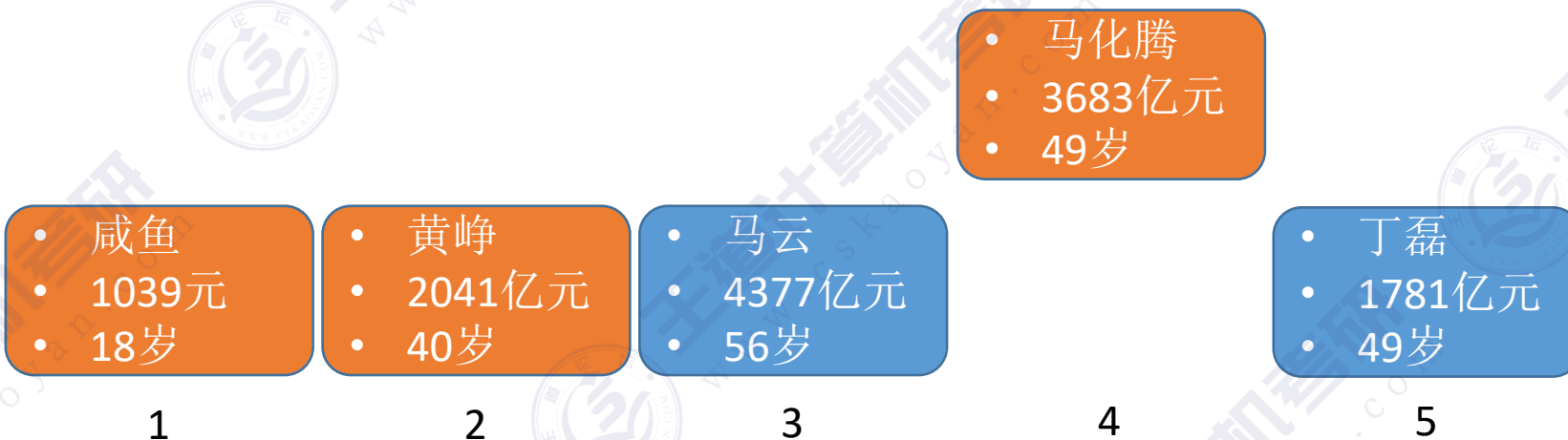
Step 2: 扫描剩下的4个元素，找到年龄最小的一个元素，插入到第2个位置

Step 3: 扫描剩下的3个元素，找到年龄最小的一个元素，插入到第3个位置

什么是算法?

程序 = 数据结构 + 算法

求解问题的
步骤



算法: 将该线性表按照年龄递增排序

Step 1: 扫描5个元素, 找到年龄最小的一个元素, 插入到第1个位置

Step 2: 扫描剩下的4个元素, 找到年龄最小的一个元素, 插入到第2个位置

Step 3: 扫描剩下的3个元素, 找到年龄最小的一个元素, 插入到第3个位置

什么是算法？

程序 = 数据结构 + 算法

求解问题的步骤

• 咸鱼 • 1039元 • 18岁	• 黄峥 • 2041亿元 • 40岁	• 马化腾 • 3683亿元 • 49岁	• 马云 • 4377亿元 • 56岁	• 丁磊 • 1781亿元 • 49岁
1	2	3	4	5

算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

Step 2: 扫描剩下的4个元素，找到年龄最小的一个元素，插入到第2个位置

Step 3: 扫描剩下的3个元素，找到年龄最小的一个元素，插入到第3个位置

Step 4: 扫描剩下的2个元素，找到年龄最小的一个元素，插入到第4个位置

什么是算法？

程序 = 数据结构 + 算法

求解问题的
步骤

- 丁磊
- 1781亿元
- 49岁

- 咸鱼
- 1039元
- 18岁

- 黄峥
- 2041亿元
- 40岁

- 马化腾
- 3683亿元
- 49岁

- 马云
- 4377亿元
- 56岁

1

2

3

4

5

算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

Step 2: 扫描剩下的4个元素，找到年龄最小的一个元素，插入到第2个位置

Step 3: 扫描剩下的3个元素，找到年龄最小的一个元素，插入到第3个位置

Step 4: 扫描剩下的2个元素，找到年龄最小的一个元素，插入到第4个位置

什么是算法？

程序 = 数据结构 + 算法

求解问题的步骤

• 咸鱼 • 1039元 • 18岁	• 黄峥 • 2041亿元 • 40岁	• 马化腾 • 3683亿元 • 49岁	• 丁磊 • 1781亿元 • 49岁	• 马云 • 4377亿元 • 56岁
1	2	3	4	5

算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

Step 2: 扫描剩下的4个元素，找到年龄最小的一个元素，插入到第2个位置

Step 3: 扫描剩下的3个元素，找到年龄最小的一个元素，插入到第3个位置

Step 4: 扫描剩下的2个元素，找到年龄最小的一个元素，插入到第4个位置

什么是算法？

程序 = 数据结构 + 算法

求解问题的
步骤

• 咸鱼 • 1039元 • 18岁	• 黄峥 • 2041亿元 • 40岁	• 马化腾 • 3683亿元 • 49岁	• 丁磊 • 1781亿元 • 49岁	• 马云 • 4377亿元 • 56岁
1	2	3	4	5

算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

Step 2: 扫描剩下的4个元素，找到年龄最小的一个元素，插入到第2个位置

Step 3: 扫描剩下的3个元素，找到年龄最小的一个元素，插入到第3个位置

Step 4: 扫描剩下的2个元素，找到年龄最小的一个元素，插入到第4个位置

算法的特性

算法必须具备的特性

有穷性。一个算法必须总在执行有穷步之后结束，且每一步都可在有穷时间内完成。

注：**算法**必须是有穷的，而**程序**可以是无穷的

用有限步骤解决某个特定的问题

如：微信是程序，不是算法

算法的特性

确定性。算法中每条指令必须有确切的含义，对于相同的输入只能得出相同的输出。

• 马云 • 4377亿元 • 56岁	• 马化腾 • 3683亿元 • 49岁	• 黄峥 • 2041亿元 • 40岁	• 丁磊 • 1781亿元 • 49岁	• 咸鱼 • 1039元 • 18岁
• 咸鱼 • 1039元 • 18岁	• 黄峥 • 2041亿元 • 40岁	• 马化腾 • 3683亿元 • 49岁	• 丁磊 • 1781亿元 • 49岁	• 马云 • 4377亿元 • 56岁
• 咸鱼 • 1039元 • 18岁	• 黄峥 • 2041亿元 • 40岁	• 丁磊 • 1781亿元 • 49岁	• 马化腾 • 3683亿元 • 49岁	• 马云 • 4377亿元 • 56岁
1	2	3	4	5

什么是算法？

程序 = 数据结构 + 算法

求解问题的
步骤

• 咸鱼 • 1039元 • 18岁	• 黄峥 • 2041亿元 • 40岁	• 马云 • 4377亿元 • 56岁	• 马化腾 • 3683亿元 • 49岁	• 丁磊 • 1781亿元 • 49岁
1	2	3	4	5

算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

Step 2: 扫描剩下的4个元素，找到年龄最小的一个元素，插入到第2个位置

Step 3: 扫描剩下的3个元素，找到年龄最小的一个元素，插入到第3个位置

Step 4: 扫描剩下的2个元素，找到年龄最小的一个元素，插入到第4个位置

算法的特性

确定性。算法中每条指令必须有确切的含义，对于相同的输入只能得出相同的输出。

• 马云 • 4377亿元 • 56岁	• 马化腾 • 3683亿元 • 49岁	• 黄峥 • 2041亿元 • 40岁	• 丁磊 • 1781亿元 • 49岁	• 咸鱼 • 1039元 • 18岁
• 咸鱼 • 1039元 • 18岁	• 黄峥 • 2041亿元 • 40岁	• 马化腾 • 3683亿元 • 49岁	• 丁磊 • 1781亿元 • 49岁	• 马云 • 4377亿元 • 56岁
• 咸鱼 • 1039元 • 18岁	• 黄峥 • 2041亿元 • 40岁	• 丁磊 • 1781亿元 • 49岁	• 马化腾 • 3683亿元 • 49岁	• 马云 • 4377亿元 • 56岁
1	2	3	4	5

可行性。算法中描述的操作都可以通过已经实现的基本运算执行有限次来实现。
输入。一个算法有零个或多个输入，这些输入取自于某个特定的对象的集合。
输出。一个算法有一个或多个输出，这些输出是与输入有着某种特定关系的量。

“好”算法的特质

设计算法时要尽量追求的目标

1) 正确性。算法应能够正确地解决求解问题。

- 马云 4377亿元 56岁	- 马化腾 3683亿元 49岁	- 黄峥 2041亿元 40岁	- 丁磊 1781亿元 49岁	- 咸鱼 1039元 18岁
1	2	3	4	5



没有正确解决问题

“好”算法的特质

2) 可读性。算法应具有良好的可读性，以帮助人们理解。

算法：将该线性表按照年龄递增排序

Step 1: 扫描5个元素，找到年龄最小的一个元素，插入到第1个位置

Step 2: 扫描剩下的4个元素，找到年龄最小的一个元素，插入到第2个位置

Step 3: 扫描剩下的3个元素，找到年龄最小的一个元素，插入到第3个位置

Step 4: 扫描剩下的2个元素，找到年龄最小的一个元素，插入到第4个位置

注：算法可以用代码、伪代码描述，甚至用文字描述，重要的是要“无歧义”地描述出解决问题的步骤



垃圾！

看不懂！



“好”算法的特质

3) 健壮性。输入非法数据时，算法能适当地做出反应或进行处理，而不会产生莫名其妙的输出结果。

• 马云 • 4377亿元 • 56岁	• 马化腾 • 3683亿元 • 49岁	• 黄峥 • 2041亿元 • 40岁	• 丁磊 • 1781亿元 • 49岁	• 咸鱼 • 1039元 • -100岁	出现非法数据
• 咸鱼 • 1039元 • -100岁	• 黄峥 • 2041亿元 • 40岁	• 马化腾 • 3683亿元 • 49岁	• 丁磊 • 1781亿元 • 49岁	• 马云 • 4377亿元 • 56岁	
1	2	3	4	5	



“好”算法的特质

4) 高效率与低存储量需求

花的时间少。
时间复杂度低

不费内存。
空间复杂度低

程序设计：
设计一个好的数据结构
设计一个好的算法

知识回顾与重要考点

数据结构是要处理的信息

什么是算法

程序 = 数据结构 + 算法

算法是处理信息的步骤

算法必须具备的特性

算法的五个特性

有穷性

有穷时间内能执行完

算法是有穷的

程序可以是无穷的

确定性

相同输入只会产生相同输出

可行性

可以用已有的基本操作实现算法

输入

丢给算法处理的数据

输出

算法处理的结果

正确性

能正确解决问题

可读性

对算法的描述要让其他人也看得懂

健壮性

算法能处理一些异常状况

"好"算法的特质

高效率与低存储量需求

即算法执行省时、省内存

时间复杂度低、空间复杂度低

设计算法时要尽量追求的目标

算法的基本概念

本节内容

算法

效率的度量

知识总览

算法效率的度量

时间复杂度

空间复杂度

如何评估算法时间开销?



让算法先运行，事后统计运行时间？



存在什么问题？

- 和机器性能有关，如：超级计算机 v.s. 单片机
- 和编程语言有关，越高级的语言执行效率越低
- 和编译程序产生的机器指令质量有关
- 有些算法是不能事后再统计的，如：导弹控制算法

能否排除与算法本身无关的外界因素

能否事先估计？



算法时间复杂度

事前预估算法时间开销 $T(n)$ 与问题规模 n 的关系（ T 表示“time”）

算法的时间复杂度



用算法表白——“爱你n遍”

算法的时间复杂度

//算法1— 逐步递增型爱你

void loveYou(int n) { //n 为问题规模

① int i=1; //爱你的程度

② while(i<=n){

③ i++; //每次+1

④ printf("I Love You %d\n", i);

}

⑤ printf("I Love You More Than %d\n", n);

}

```
int main(){
    loveYou(3000);
}
```

I Love You 2994

I Love You 2995

I Love You 2996

I Love You 2997

I Love You 2998

I Love You 2999

I Love You 3000

I Love You 3001

I Love You More Than 3000

语句频度:

① ——1次

② ——3001次

③④ ——3000次

⑤ ——1次

$T(3000) = 1 + 3001 + 2 * 3000 + 1$

时间开销与问题规模 n 的关系:

$T(n) = 3n + 3$

思考中.....



问题1: 是否可以忽略表达式某些部分?

问题2: 如果有好几千行代码, 按这种方法需要一行一行数?

算法的时间复杂度

全称：渐进时间复杂度

思考中.....



问题1：是否可以忽略表达式某些部分？

当问题规模 n 足够大时...

大O表示“同阶”，同等数量级。即：当 $n \rightarrow \infty$ 时，二者之比为常数

$$\begin{aligned}T_1(n) &= O(n) \\T_2(n) &= O(n^2) \\T_3(n) &= O(n^3)\end{aligned}$$

简化

时间开销与问题规模 n 的关系：

$$T_1(n) = 3n + 3$$

$$T_2(n) = n^2 + 3n + 1000$$

$$T_3(n) = n^3 + n^2 + 99999999$$

若 $n=3000$ ，则

$$3n = 9000$$

$$n^2 = 9,000,000$$

$$n^3 = 27,000,000,000$$

V.S.

$$T_1(n) = 9003$$

V.S.

$$T_2(n) = 9,010,000$$

V.S.

$$T_3(n) = 27,018,999,999$$

结论1：可以只考虑阶数高的部分

当 $n=3000$ 时， $9999n = 29,997,000$ 远小于 $n^3 = 27,018,999,999$
当 $n=1000000$ 时， $9999n = 9,999,000,000$ 远小于 $n^2 = 1,000,000,000,000$

结论2：问题规模足够大时，常数项系数也可以忽略

算法的时间复杂度

思考中.....



问题1: 是否可以忽略表达式某些部分?

当问题规模 n 足够大时...

时间开销与问题规模 n 的关系:

$$T_1(n) = 3n + 3$$

$$T_2(n) = n^2 + 3n + 1000$$

$$T_3(n) = n^3 + n^2 + 99999999$$

$$T_1(n) = O(n)$$

$$T_2(n) = O(n^2)$$

$$T_3(n) = O(n^3)$$

简化

a) 加法规则

$$T(n) = T_1(n) + T_2(n) = O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

多项相加, 只保留最高阶的项, 且系数变为1

b) 乘法规则

$$T(n) = T_1(n) \times T_2(n) = O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$$

多项相乘, 都保留

$$\begin{aligned} \text{Eg: } T_3(n) &= n^3 + n^2 \log_2 n \\ &= O(n^3) + O(n^2 \log_2 n) \\ &= ? ? ? \end{aligned}$$

算法的时间复杂度

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$



问题：两个算法的时间复杂度分别如下，哪个的阶数更高（时间复杂度更高）？

$$T_1(n) = O(n)$$

$$T_2(n) = O(\log_2 n)$$

$$\begin{aligned} \text{Eg: } T_3(n) &= n^3 + n^2 \log_2 n \\ &= O(n^3) + O(n^2 \log_2 n) \\ &= O(n^2 n) + O(n^2 \log_2 n) \end{aligned}$$

$$\lim_{n \rightarrow \infty} \frac{\log_2 n}{n} \xrightarrow{\text{洛必达}} \lim_{n \rightarrow \infty} \frac{1}{n \ln 2} = 0$$

当 $n \rightarrow \infty$ 时， n 比 $\log_2 n$ 变大的速度快很多

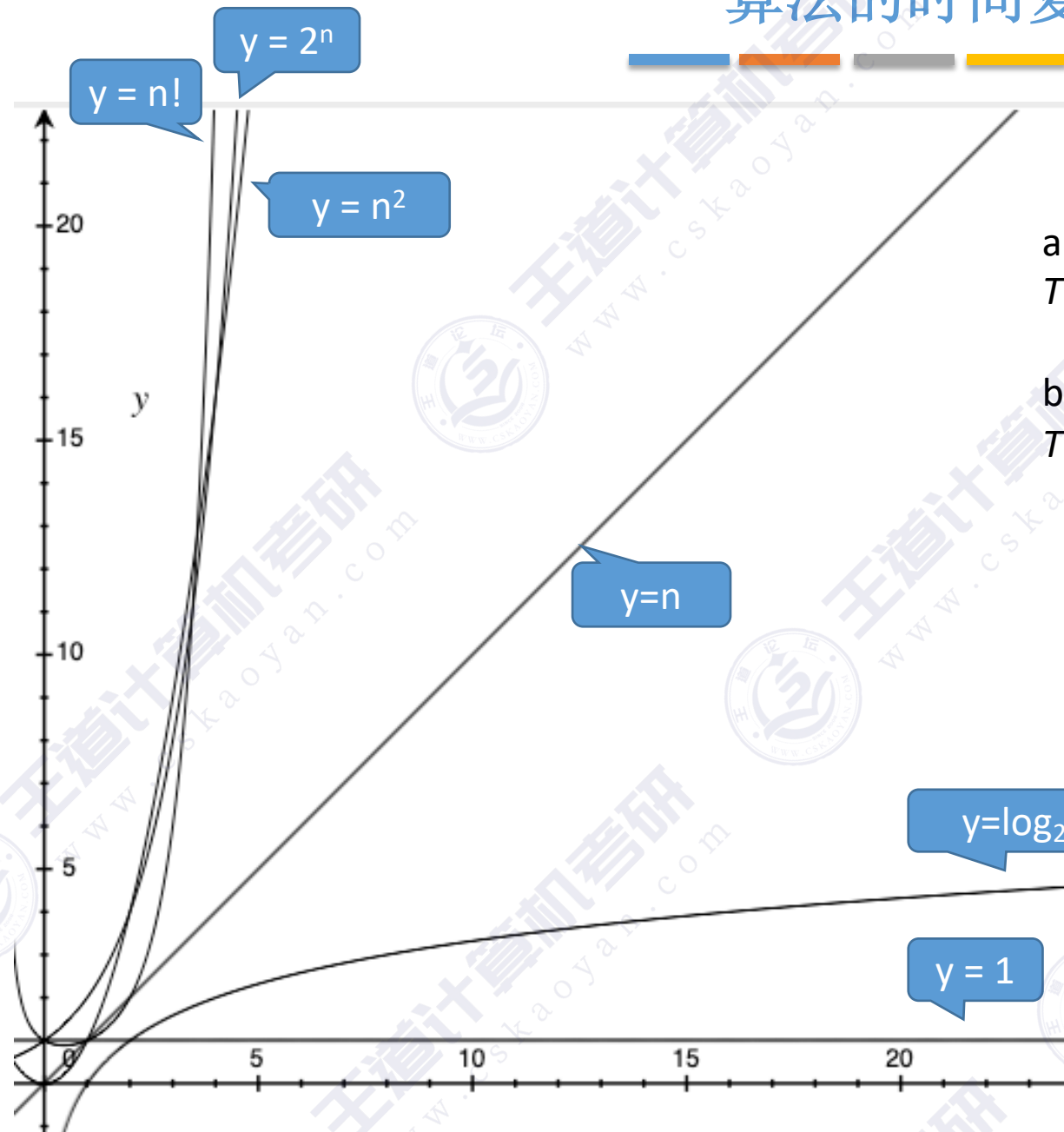
$$\lim_{n \rightarrow \infty} \frac{n^2}{2^n} \xrightarrow{\text{洛必达}} \lim_{n \rightarrow \infty} \frac{2n}{2^n \ln 2} \xrightarrow{\text{洛必达}} \lim_{n \rightarrow \infty} \frac{2}{2^n \ln^2 2} = 0$$

当 $n \rightarrow \infty$ 时， 2^n 比 n^2 变大的速度快很多

别紧张 放轻松 😊



算法的时间复杂度



只保留更高阶的项

a) 加法规则

$$T(n) = T_1(n) + T_2(n) = O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

b) 乘法规则

$$T(n) = T_1(n) \times T_2(n) = O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$$



巴拉拉能量——

常对幂指阶

数据结构 +20分

算法的时间复杂度

//算法1— 逐步递增型爱你

void loveYou(int n) { //n 为问题规模

① int i=1; //爱你的程度

② while(i<=n){

③ i++; //每次+1

④ printf("I Love You %d\n", i);

}

⑤ printf("I Love You More Than %d\n", n);

}

```
int main(){
    loveYou(3000);
}
```

I Love You 2994

I Love You 2995

I Love You 2996

I Love You 2997

I Love You 2998

I Love You 2999

I Love You 3000

I Love You 3001

I Love You More Than 3000

语句频度:

① ——1次

② ——3001次

③④ ——3000次

⑤ ——1次

$T(3000) = 1 + 3001 + 2 * 3000 + 1$

时间开销与问题规模 n 的关系:

$T(n) = 3n + 3 = O(n)$

思考中.....



问题1: 是否可以忽略表达式某些部分?

只考虑阶数, 用大O记法表示

问题2: 如果有好几千行代码, 按这种方法需要一行一行数?

算法的时间复杂度

//算法1— 逐步递增型爱你

void loveYou(int n) { //n 为问题规模

```
① int i=1;    //爱你的程度
② while(i<=n){
③     i++;    //每次+1
④     printf("I Love You %d\n", i);
}
⑤ printf("I Love You More Than %d\n", n);
}
```

此处插入1000行顺序执行的代码

语句频度:

- ① ——1次
- ② ——3001次
- ③④ ——3000次
- ⑤ ——1次

$T(3000) = 1 + 3001 + 2 * 3000 + 1$

时间开销与问题规模 n 的关系:

$$T(n) = 3n + 3 = O(n)$$

结论1: 顺序执行的代码只会影响常数项, 可以忽略

结论2: 只需挑循环中的一个基本操作分析它的执行次数与 n 的关系即可



机智如我

$T(3000) = 1000 + 1 + 3001 + 2 * 3000 + 1$

时间开销与问题规模 n 的关系:

$$T(n) = 3n + 1003 = O(n)$$

算法的时间复杂度

//算法2— 嵌套循环型爱你

```
void loveYou(int n) { //n 为问题规模
    int i=1; //爱你的程度
    while(i<=n){
        i++; //每次+1
        printf("I Love You %d\n", i);
        for (int j=1; j<=n; j++){
            printf("I am Iron Man\n");
        }
    }
    printf("I Love You More Than %d\n", n);
}
```

外层循环执行n次

嵌套两层循环

内层循环共执行 n^2 次

时间开销与问题规模 n 的关系:

$$T(n) = O(n) + O(n^2) = O(n^2)$$

结论1: 顺序执行的代码只会影响常数项, 可以忽略

结论2: 只需挑循环中的一个基本操作分析它的执行次数与 n 的关系即可

结论3: 如果有多层嵌套循环, 只需关注最深层循环循环了几次



机智如我

算法的时间复杂度

//算法1— 逐步递增型爱你

void loveYou(int n) { //n 为问题规模

① int i=1; //爱你的程度

② while(i<=n){

③ i++; //每次+1

④ printf("I Love You %d\n", i);

}

⑤ printf("I Love You More Than %d\n", n);

}

```
int main(){
    loveYou(3000);
}
```

I Love You 2994

I Love You 2995

I Love You 2996

I Love You 2997

I Love You 2998

I Love You 2999

I Love You 3000

I Love You 3001

I Love You More Than 3000

语句频度:

① ——1次

② ——3001次

③④ ——3000次

⑤ ——1次

$T(3000) = 1 + 3001 + 2 * 3000 + 1$

时间开销与问题规模 n 的关系:

$T(n) = 3n + 3 = O(n)$

思考中.....



问题1: 是否可以忽略表达式某些部分?

只考虑阶数, 用大O记法表示

问题2: 如果有好几千行代码, 按这种方法需要一行一行数?

只需考虑最深层循环的循环次数与 n 的关系

小练习1

//算法3— 指数递增型爱你

```
void loveYou(int n) { //n 为问题规模
    int i=1; //爱你的程度
    while(i<=n){
        i=i*2; //每次翻倍
        printf("I Love You %d\n", i);
    }
    printf("I Love You More Than %d\n", n);
}
```

```
I Love You 32
I Love You 64
I Love You 128
I Love You 256
I Love You 512
I Love You 1024
I Love You 2048
I Love You 4096
I Love You More Than 3000
```

计算上述算法的时间复杂度 $T(n)$:

设最深层循环的语句频度（总共循环的次数）为 x ，则由循环条件可知，循环结束时刚好满足 $2^x > n$

$$x = \log_2 n + 1$$

$$T(n) = O(x) = O(\log_2 n)$$

小练习2

//算法4— 搜索数字型爱你

```
void loveYou(int flag[], int n) { //n 为问题规模
    printf("I Am Iron Man\n");
    for(int i=0; i<n; i++){ //从第一个元素开始查找
        if(flag[i]==n){ //找到元素n
            printf("I Love You %d\n", n);
            break; //找到后立即跳出循环
        }
    }
}
```

//flag 数组中乱序存放了 1~n 这些数

```
int flag[n]={1...n};
loveYou(flag, n);
```

计算上述算法的时间复杂度 $T(n)$

最好情况：元素n在第一个位置

最坏情况：元素n在最后一个位置

平均情况：假设元素n在任意一个位置的概率相同为 $\frac{1}{n}$

很多算法执行时间与输入的数据有关

——最好时间复杂度 $T(n)=O(1)$

——最坏时间复杂度 $T(n)=O(n)$

——平均时间复杂度 $T(n)=O(n)$

$$\text{循环次数 } x = (1+2+3+\dots+n) \frac{1}{n} = \left(\frac{n(1+n)}{2}\right) \frac{1}{n} = \frac{1+n}{2} \quad T(n)=O(x)=O(n)$$

算法的时间复杂度



最坏时间复杂度：最坏情况下算法的时间复杂度

平均时间复杂度：所有输入示例等概率出现的情况下，算法的期望运行时间

最好时间复杂度：最好情况下算法的时间复杂度

知识回顾与重要考点

时间复杂度

如何计算

① 找到一个基本操作（最深层循环）

② 分析该基本操作的执行次数 x 与问题规模 n 的关系 $x=f(n)$

③ x 的数量级 $O(x)$ 就是算法时间复杂度 $T(n)$

加法规则: $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$

乘法规则: $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$

常用技巧

“常对幂指阶”

$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$

三种复杂度

最坏时间复杂度: 考虑输入数据“最坏”的情况

平均时间复杂度: 考虑所有输入数据都等概率出现的情况

最好时间复杂度: 考虑输入数据“最好”的情况

小故事: 算法的性能问题只有在 n 很大时才会暴露出来。

本节内容

算法

效率的度量

知识总览

算法效率的度量

时间复杂度

时间开销与问题规模 n 之间的关系

空间复杂度

空间开销（内存开销）与问题规模 n 之间的关系

程序运行时的内存需求

```
//算法1— 逐步递增型爱你
void loveYou(int n) { //n 为问题规模
    int i=1; //爱你的程度
    while(i<=n){
        i++; //每次+1
        printf("I Love You %d\n", i);
    }
    printf("I Love You More Than %d\n", n);
}
```

装入

内存

程序
代码

大小固定，与
问题规模无关

数据

局部变量 i,
参数 n ...

无论问题规模怎么变，算法运行所需的内存空间都是固定的常量，算法空间复杂度为

$$S(n) = O(1)$$

注：S 表示 “Space”

算法原地工作——算法所需内存空间为常量

空间复杂度

```
void test(int n) {  
    int flag[n];    //声明一个长度为n的数组  
    int i;  
    //.....此处省略很多代码  
}
```

装入

内存

程序
代码

大小固定，与
问题规模无关

数据

局部变量 i，
参数 n ...
数组 flag[n]

假设一个 int 变量占 4B...

则所需内存空间 = 4 + 4n + 4 = 4n + 8

$$S(n) = O(n)$$

只需关注存储空间大小
与问题规模相关的变量

空间复杂度

```
void test(int n) {  
    int flag[n][n]; //声明 n*n 的二维数组  
    int i;  
    //.....此处省略很多代码  
}
```

装入

内存

程序
代码

大小固定，与
问题规模无关

数据

局部变量 i，
参数 n ...
数组 flag[n][n]

$$S(n) = O(n^2)$$

空间复杂度

```
void test(int n) {  
    int flag[n][n]; //声明 n*n 的二维数组  
    int other[n];   //声明一个长度为n的数组  
    int i;  
    //.....此处省略很多代码  
}
```

装入

内存

程序
代码

大小固定，与
问题规模无关

数据

局部变量 i,
参数 n ...
数组 flag[n][n]
数组 other[n]

$$S(n) = O(n^2) + O(n) + O(1) = O(n^2)$$

a) 加法规则

$$T(n) = T_1(n) + T_2(n) = O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

函数递归调用带来的内存开销

```
//算法5— 递归型爱你
void loveYou(int n) {    //n 为问题规模
    int a,b,c;  //声明一系列局部变量
    //...省略代码
    if (n > 1) {
        loveYou(n-1);
    }
    printf("I Love You %d\n", n);
}
```

```
int main(){
    loveYou(5);
}
```

```
I Love You 1
I Love You 2
I Love You 3
I Love You 4
I Love You 5
```

装入

内存

程序
代码

大小固定，与
问题规模无关

存储函数的参数n，
局部变量abc...

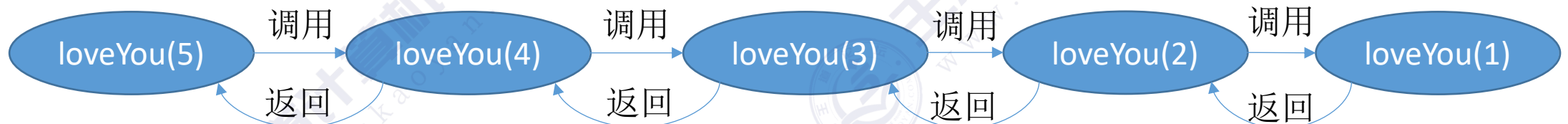
5

4

3

2

1



函数递归调用带来的内存开销

```
//算法5— 递归型爱你
void loveYou(int n) {    //n 为问题规模
    int a,b,c;  //声明一系列局部变量
    //...省略代码
    if (n > 1) {
        loveYou(n-1);
    }
    printf("I Love You %d\n", n);
}
```

```
int main(){
    loveYou(5);
}
```

```
I Love You 1
I Love You 2
I Love You 3
I Love You 4
I Love You 5
```

装入

内存

程序
代码

大小固定，与
问题规模无关

存储函数的参数n，
局部变量abc...

5

4

3

2

1

$$S(n) = O(n)$$

空间复杂度 = 递归调用的深度



函数递归调用带来的内存开销

//算法5— 递归型爱你

```
void loveYou(int n) { //n 为问题规模
    int flag[n];      //声明一个数组
    //...省略数组初始化代码
    if (n > 1) {
        loveYou(n-1);
    }
    printf("I Love You %d\n", n);
}
```

```
int main(){
    loveYou(5);
}
```

```
I Love You 1
I Love You 2
I Love You 3
I Love You 4
I Love You 5
```

装入

内存

程序
代码

大小固定，与
问题规模无关

5

存储参数 n，flag[5]...

4

存储参数 n，flag[4]...

3

2

1

存储参数 n，flag[1]...

$$1+2+3+\dots+n = [n(1+n)]/2 = \frac{1}{2}n^2 + \frac{1}{2}n$$

$$S(n) = O(n^2)$$

知识回顾与重要考点

空间复杂度

如何计算

普通程序

- ① 找到所占空间大小与问题规模相关的变量
- ② 分析所占空间 x 与问题规模 n 的关系 $x=f(n)$
- ③ x 的数量级 $O(x)$ 就是算法空间复杂度 $S(n)$

递归程序

- ① 找到递归调用的深度 x 与问题规模 n 的关系 $x=f(n)$
- ② x 的数量级 $O(x)$ 就是算法空间复杂度 $S(n)$

注：有的算法各层函数所需存储空间不同，分析方法略有区别

常用技巧

加法规则： $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$

乘法规则： $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$

“常对幂指阶”

$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$